Rain- Based Traffic Alert System

Team No: 6

Team Name: Nova

Team Members:

1. Manisha N - 2360412
2. Indla Sai Sneha - 2362080
3. Reyona - 2360601
4. Libena Igi J – 2360519

MileStone 2: Dataset Generation and ML model Development

1. Introduction

The second milestone focuses on building the data-driven component of the Intelligent Rain-Based Traffic Alert System. This includes generating a dataset from simulated sensor values and applying machine learning techniques to classify road safety conditions. The goal is to improve prediction accuracy and enable intelligent decision-making.

2. Dataset Generation

The dataset is synthetically generated using simulated sensor values obtained from the IoT system. Since real-time large-scale data collection is not feasible at this stage, random data generation is used to create multiple scenarios.

Features Used:

- Rain (0-4095) - Rain intensity
- Traffic (0 or 1) - Vehicle presence
- Distance (cm) - Distance between vehicles
- Temperature (°C) - Environmental condition
- Humidity(%) - Moisture level in air
- Visibility (low/medium/high or numeric)- Road visibility
- Speed(km/h) - Vehicle speed
- Risk - Output label (SAFE, CAUTION, HIGH, CRITICAL)

2.1 Dataset Creation Method

- Random values are generated for each feature
- Different combinations simulate real-world conditions

- Each data point is labeled based on predefined rules
- Dataset is stored in CSV format for ML processing

2.2 Head of Dataset

```
Total rows: 1500

Sample data:
   Rain  Traffic  Distance  Temperature  Humidity  Visibility  Speed  Risk
0   964        0       195           21        52         100     24  Safe
1  2582        1        94           35        41          16     27  High
2  3663        1        56           40        40          95     75  High
3  3464        0        64           22        95          28    100  High
4  3298        1       109           40        92          70     48  High
```

3. Risk Classification Logic

1. CRITICAL : It combines High rain , traffic present and very low distance
2. HIGH : Medium rain or moderate distance
3. CAUTION : Low rain and safe distance
4. SAFE : No traffic and normal conditions

4. Machine Learning Model

Model Used: Random Forest

Used RandomForest for several reasons:

- Ability to Measure Feature Importance
- Works well with structured data
- Easy to interpret
- Suitable for classification problems
- Handling Messy and High-Dimensional Data

4.1 Model Workflow

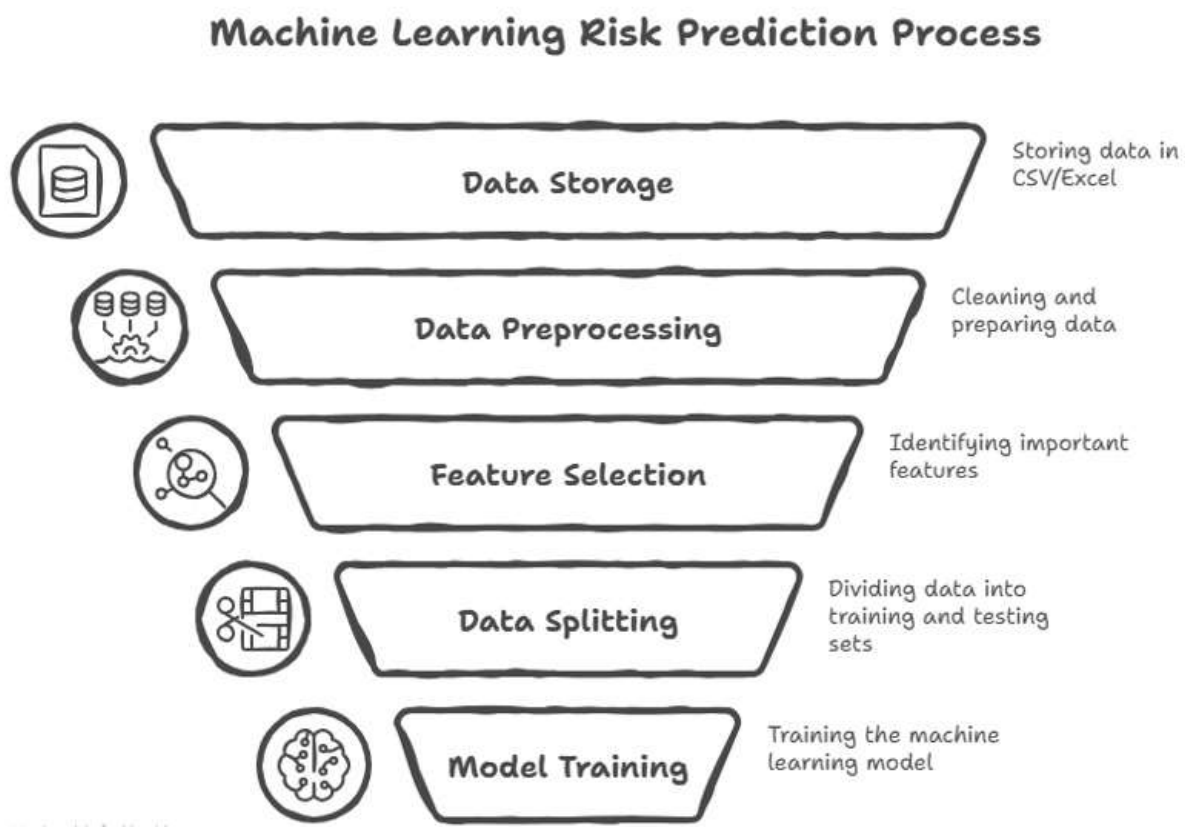
1. Load dataset
2. Split into training and testing data
3. Train ML model
4. Predict risk levels

5. Evaluate accuracy

6. Data Storage in Excel

The generated dataset is stored in an Excel format to enable easy visualization, analysis, and sharing. Excel is used because it provides a simple tabular structure and supports integration with machine learning tools.

Workflow



7. Code

7.1 DataSet Creation and Storing in Excel Automatically

```
import pandas as pd

import random

num_rows = 1500

data = []

for i in range(num_rows):

    rain = random.randint(0, 4095)

    traffic = random.randint(0, 1)

    distance = random.randint(5, 200)
```

```
    temperature = random.randint(20, 40)
    humidity = random.randint(40, 100)
    visibility = random.randint(10, 100)
    speed = random.randint(20, 100)
    if rain > 3000 and distance < 20 and traffic == 1:
        risk = "Critical"
    elif rain > 2000 or distance < 40 or visibility < 30:
        risk = "High"
    elif rain > 1000 or speed > 80:
        risk = "Caution"
    else:
        risk = "Safe"
    data.append([
        rain, traffic, distance, temperature,
        humidity, visibility, speed, risk
    ])
df = pd.DataFrame(data, columns=[
    "Rain",
    "Traffic",
    "Distance",
    "Temperature",
    "Humidity",
    "Visibility",
    "Speed",
    "Risk"
])
df.to_excel("rain_traffic_dataset.xlsx", index=False)
df.to_csv("rain_traffic_dataset.csv", index=False)
print("Dataset successfully created!")
```

```
print("Total rows:", len(df))
print("\nSample data:")
print(df.head())
```

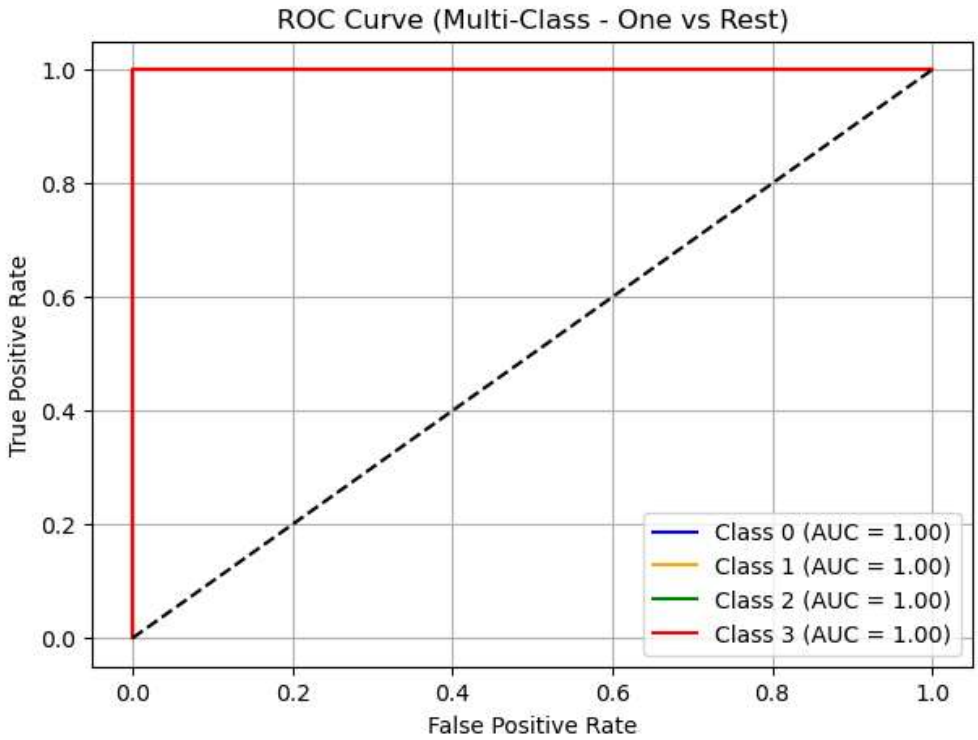
7.2 Model Training

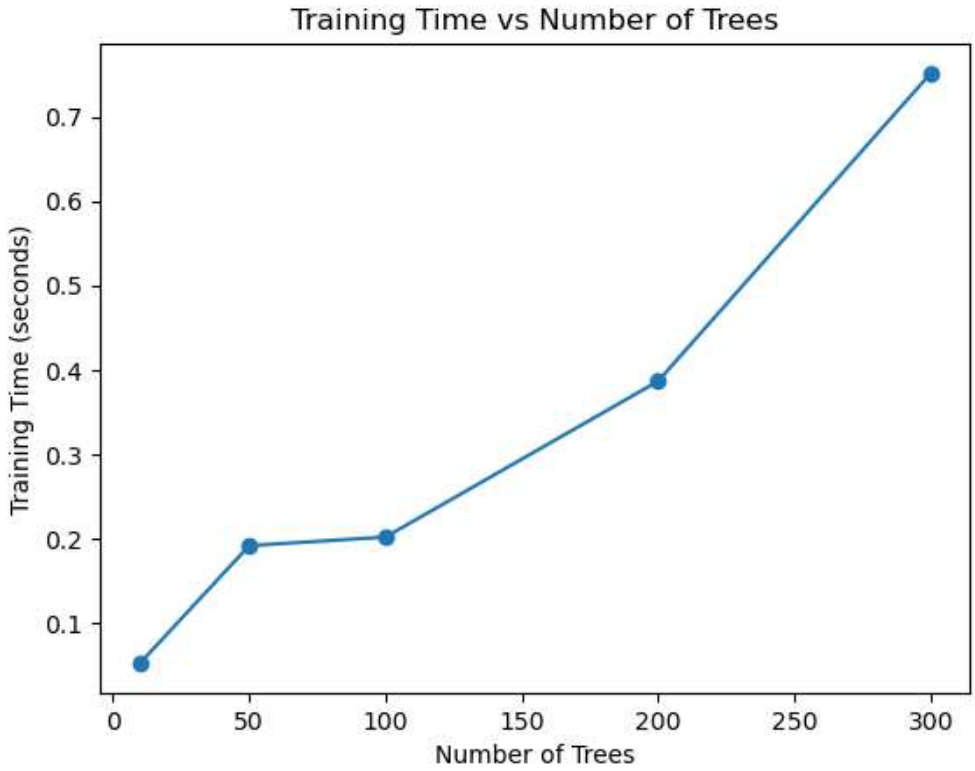
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import LabelEncoder
import joblib
df = pd.read_excel("rain_traffic_dataset.xlsx")
print("Dataset Loaded!")
print(df.head())
le = LabelEncoder()
df['Risk'] = le.fit_transform(df['Risk'])
print("\nRisk Classes:", list(le.classes_))
X = df[['Rain', 'Traffic', 'Distance', 'Temperature',
        'Humidity', 'Visibility', 'Speed']]
y = df['Risk']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = RandomForestClassifier(
    n_estimators=300,
    max_depth=15,
    min_samples_split=2,
    min_samples_leaf=1,
    random_state=42
)
```

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("\n MODEL TRAINED")
print("Accuracy: {:.2f}%".format(accuracy * 100))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
joblib.dump(model, "high_accuracy_model.pkl")
print("\nModel saved as high_accuracy_model.pkl")
sample = [[3500, 1, 10, 30, 80, 20, 90]]
prediction = model.predict(sample)
print("\nSample Prediction (Encoded):", prediction)
print("Predicted Risk:", le.inverse_transform(prediction))
```

7.3 Results and Output

```
Dataset Loaded!
   Rain  Traffic  Distance  Temperature  Humidity  Visibility  Speed  Risk
0   964        0       195           21        52         100     24  Safe
1  2582        1        94           35        41          16     27  High
2  3663        1        56           40        40          95     75  High
3  3464        0        64           22        95          28    100  High
4  3298        1       109           40        92          70     48  High

Risk Classes: ['Caution', 'Critical', 'High', 'Safe']

 MODEL TRAINED
Accuracy: 99.33%

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        55
           1       1.00      0.33      0.50         3
           2       0.99      1.00      1.00       203
           3       1.00      1.00      1.00        39

    accuracy                           0.99       300
   macro avg       1.00      0.83      0.87       300
weighted avg       0.99      0.99      0.99       300
...
Model saved as high_accuracy_model.pkl

Sample Prediction (Encoded): [1]
Predicted Risk: ['Critical']
```


ROC Curve (Multi-Class - One vs Rest)
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Class 0 (AUC = 1.00)
Class 1 (AUC = 1.00)
Class 2 (AUC = 1.00)
Class 3 (AUC = 1.00)


Training Time vs Number of Trees
Training Time (seconds)
Number of Trees
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0
50
100
150
200
250
300

6. Conclusion

The second milestone shows how our system uses data to predict traffic risk during rain. We created a dataset using simulated sensor values like rain, traffic, distance, temperature, humidity, visibility, and speed to represent real-world conditions. This data was stored in an Excel format so it can be easily viewed and used. We then used simple rules to classify the situation into four levels: safe, caution, high, and critical. Adding features like visibility, humidity, and speed makes the system more accurate and realistic. The workflow shows how data is generated, processed, and used to make predictions. Overall, this milestone builds a strong base for creating a smart traffic alert system that can be improved further using real-time data and advanced machine learning in the future.